코딩테스트 개념노트: 쉽게 풀어쓴 학습판

> 대상: 자바스크립트 문법은 알지만 알고리즘의 원리와 문제 적용이 어려운 학습자
> 구성: 한 줄 핵심 → 손으로 따라가기 → 실행 가능한 코드 → 연습문제와 풀이
> 참고: 제공된 PDF《[개념 노트] 자바스크립트로 배우는 자료구조와 알고리즘》(인프런 Kyo, https://inf.run/G9gF7)을 바탕으로 새로 설명하고 예제를 작성했다. 원본의 부정확하거나 혼동하기 쉬운 설명은 바로잡았다.

이 노트 사용법

1. 먼저 **한 줄 핵심**만 읽고 문제를 떠올린다.
2. **손으로 따라가기**에서 값이 어떻게 바뀌는지 확인한다.
3. 코드의 `console.assert`를 Node.js로 실행한다. `node 파일명.js`에서 오류가 없으면 통과다.
4. 연습문제를 직접 풀고 풀이를 본다. 코드 암기보다 **왜 이 자료구조나 알고리즘을 쓰는지** 설명할 수 있는지가 중요하다.

복잡도에서 `n`은 보통 입력 크기, `V`는 정점 수, `E`는 간선 수다. 별도 설명이 없으면 시간 복잡도는 최악의 경우를 뜻한다.

1. 스택과 큐

한 줄 핵심

- **스택**: 마지막에 넣은 것이 먼저 나온다. 접시 더미와 같다. `push`와 `pop`.
- **큐**: 먼저 넣은 것이 먼저 나온다. 줄 서기와 같다. `enqueue`와 `dequeue`.

손으로 따라가기

`A`, `B`, `C`를 차례로 넣으면 스택은 `[A, B, C]`에서 `C`를 먼저 꺼낸다. 큐는 같은 순서로 넣어도 `A`를 먼저 꺼낸다. **괄호 짝 확인**은 최근에 열린 괄호를 먼저 닫으므로 스택, **가까운 곳부터 탐색**하는 BFS는 먼저 발견한 곳부터 처리하므로 큐다.

실행 코드

```
// 스택: 배열 끝에서 넣고 뺀다.
const stack = [];
stack.push('A', 'B', 'C');
console.assert(stack.at(-1) === 'C'); // peek: 꺼내지 않고 보기
console.assert(stack.pop() === 'C');
console.assert(stack.pop() === 'B');

// 큐: 앞쪽 원소를 매번 shift하지 않고 읽기 위치만 옮긴다.
const queue = ['A', 'B', 'C'];
let head = 0;
console.assert(queue[head++] === 'A');
console.assert(queue[head++] === 'B');
queue.push('D');
console.assert(queue[head++] === 'C');
```

배열의 `push` / `pop` 은 끝에서 처리하므로 보통 `O(1)` 이다. `shift()` 는 나머지 원소를 옮길 수 있어 한 번에 `O(n)` 이다. 코딩테스트 큐는 `head` 인덱스를 쓰면 각 꺼내기가 `O(1)` 이다. 오랫동안 운영하는 서비스라면 지나간 앞부분을 정리해야 하지만, 한 번 실행하는 탐색 문제에서는 보통 필요 없다.

연습문제

`push(1) → push(2) → pop() → push(3) → pop()` 의 두 `pop` 결과는?

풀이: 첫 `pop` 은 2, 두 번째는 3. 스택에는 마지막에 1이 남는다.

2. 단일 연결 리스트

한 줄 핵심

각 노드가 **값과 다음 노드 참조**를 가진다. `A → B → C → null`.

손으로 따라가기

`B` 뒤에 `X` 를 넣으려면 `X.next = B.next`, 그다음 `B.next = X` 다. 순서를 거꾸로 하면 원래 `C` 로 가는 길을 잃는다. `B` 다음 노드를 지우려면 `B.next = B.next.next` 다. **노드 위치를 이미 알고 있을 때** 연결 변경은 `O(1)` 이지만, 값을 찾아 그 노드까지 가는 시간은 `O(n)` 이다.

실행 코드

```
class Node {
  constructor(value, next = null) {
    this.value = value;
    this.next = next;
  }
}

let head = new Node('A', new Node('B', new Node('C')));

function insertAfter(node, value) {
  node.next = new Node(value, node.next);
}

function removeAfter(node) {
  if (node.next === null) return undefined;
  const removed = node.next;
  node.next = removed.next;
  return removed.value;
}

insertAfter(head.next, 'X'); // A → B → X → C
console.assert(head.next.next.value === 'X');
console.assert(removeAfter(head.next) === 'X'); // A → B → C
console.assert(head.next.next.value === 'C');
head = new Node('H', head); // 맨 앞 삽입은 head를 바꾼다.
console.assert(head.value === 'H');
```

주의: 자바스크립트 배열은 길이를 바꿀 수 있다. 원본 노트의 "배열 크기를 한 번 정하면 못 바꾼다"는 설명을 JS 배열에 그대로 적용하면 틀린다. 연결 리스트의 장점은 **알고 있는 노드 근처의 연결을 적게 바꾸는 것**이지, 언제나 배열보다 빠르다는 뜻이 아니다. 인덱스로 바로 읽는 배열은 `O(1)`이고 연결 리스트는 `O(n)`이다.

연습문제

`A → B → C`에서 `A` 다음 노드를 삭제하면 무엇이 남는가? 시간은?

풀이: `A.next = A.next.next`이므로 `A → C`. `A` 참조를 이미 갖고 있다면 `O(1)`이다.

3. 이중 연결 리스트

한 줄 핵심

노드가 `next`와 `prev`를 모두 가진다. 양방향 이동과 **이미 알고 있는 노드의 삭제**가 쉽다.

손으로 따라가기

`A ⇄ B ⇄ C`에서 `B`를 제거하면 `A.next = C`, `C.prev = A`다. 연결 **양쪽**을 모두 고쳐야 한다. 단일 연결 리스트에서는 `B`의 앞 노드 `A`를 찾아야 했지만, 이중 연결 리스트에서는 `B.prev`로 바로 간다. 첫 노드·마지막 노드는 한쪽 이웃이 없으므로 `null`을 확인한다.

실행 코드

```
class DNode {
  constructor(value) {
    this.value = value;
    this.prev = null;
    this.next = null;
  }
}

function insertAfter(node, value) {
  const added = new DNode(value);
  added.prev = node;
  added.next = node.next;
  if (node.next) node.next.prev = added;
  node.next = added;
  return added;
}

function remove(node) {
  if (node.prev) node.prev.next = node.next;
  if (node.next) node.next.prev = node.prev;
  node.prev = node.next = null;
}

const a = new DNode('A');
const b = insertAfter(a, 'B');
const c = insertAfter(b, 'C');
remove(b);
console.assert(a.next === c && c.prev === a);
```

위 예시는 중간 노드 삭제를 보여준다. 실제 리스트 객체를 만든다면 맨 앞을 지울 때 `head`, 맨 뒤를 지울 때 `tail`도 갱신해야 한다. `prev` 저장 공간이 추가로 든다.

연습문제

`A ⇄ B ⇄ C`에서 `C` 뒤에 `D`를 넣을 때 바뀌는 참조는?

풀이: `D.prev = C`, `D.next = null`, `C.next = D`. 리스트가 `tail`을 관리한다면 `tail = D`도 필요하다.

먼저 알아둘 것: 시간 복잡도와 빅오

한 줄 핵심

빅오는 입력이 커질 때 **연산 횟수가 얼마나 빨리 늘어나는지**를 비교한다. 실제 초 단위 실행 시간은 아니다.

손으로 따라가기

코드의 모양	횟수 감각	복잡도
배열 한 칸 읽기	거의 일정	`O(1)`
절반씩 범위 줄이기	`16 → 8 → 4 → 2 → 1`	`O(log n)`
원소 한 번씩 보기	`n`번	`O(n)`
매 단계 전체를 훑으며 반으로 나누기	`n × log n`	`O(n log n)`
모든 원소 쌍 확인	대략 `n²/2`번	`O(n²)`

`n = 1,000`이면 `n²`은 약 백만 단계지만 `n log₂ n`은 약 만 단계다. 상수와 작은 입력에서는 실제 시간이 다를 수 있으므로 빅오만으로 실행 시간을 단정하지 않는다.

실행 코드

```
function sum(arr) {
  let total = 0;
  for (const value of arr) total += value; // n개 각각 한 번
  return total; // O(n) 시간, O(1) 추가 공간
}

function containsPairWithSameValue(arr) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] === arr[j]) return true;
    }
  }
  return false; // 최악 O(n²)
}

console.assert(sum([1, 2, 3]) === 6);
console.assert(containsPairWithSameValue([4, 2, 4]));
```

복잡도는 **코드 전체의 실제 연산**을 보고 판단한다. 원본 노트의 병합 정렬 예시는 병합 중 `shift()`를 반복한다. JS 배열의 `shift()`가 원소를 옮길 수 있어서 그 구현을 무조건 `O(n log n)`이라 보기 어렵다. 아래 5장에서는 인덱스로 병합한다.

연습문제

길이 `n` 배열을 두 번 **따로** 순회하면 `O(n)`인가, `O(n²)`인가?

풀이: `n + n = 2n`이므로 `O(n)`. 중첩해서 각 원소마다 전체를 순회해야 `O(n²)`이다.

4. 정렬

한 줄 핵심

정렬은 값을 순서대로 놓는 일이다. 학습용 정렬 세 가지는 **비교·교환 위치**가 다르고, 병합·퀵 정렬은 큰 문제를 나눈다. 실전에서 단순 정렬이면 먼저 JS 내장 `sort`를 고려한다.

손으로 따라가기

`[3, 1, 2]`를 오름차순 정렬한다.

- **버블:** 이웃끼리 비교. `3↔1` → `[1,3,2]`, `3↔2` → `[1,2,3]`. 큰 값이 끝으로 간다.
- **선택:** 전체에서 최솟값 `1`을 골라 첫 칸과 교환 → `[1,3,2]`; 남은 부분의 최솟값 `2`를 골라 교환.
- **삽입:** 앞쪽을 정렬된 구간으로 본다. `1`을 `3` 앞에, `2`를 `1`과 `3` 사이에 넣는다.
- **병합:** `[3]`과 `[1,2]`로 나눠 각각 정렬한 뒤, 앞 원소끼리 비교하며 합친다.
- **퀵:** 피벗을 정해 작은 값은 왼쪽, 큰 값은 오른쪽으로 보내고 각 부분을 다시 정렬한다.

실행 코드

```
function bubbleSort(input) {
  const a = [...input];
  for (let end = a.length - 1; end > 0; end--) {
    let changed = false;
    for (let i = 0; i < end; i++) {
      if (a[i] > a[i + 1]) {
        [a[i], a[i + 1]] = [a[i + 1], a[i]];
        changed = true;
      }
    }
    if (!changed) break;
  }
  return a;
}

function selectionSort(input) {
  const a = [...input];
  for (let i = 0; i < a.length; i++) {
    let min = i;
    for (let j = i + 1; j < a.length; j++) {
      if (a[j] < a[min]) min = j;
    }
    [a[i], a[min]] = [a[min], a[i]];
  }
  return a;
}

function insertionSort(input) {
  const a = [...input];
  for (let i = 1; i < a.length; i++) {
    const value = a[i];
    let j = i - 1;
    while (j >= 0 && a[j] > value) {
      a[j + 1] = a[j];
      j--;
    }
    a[j + 1] = value;
  }
  return a;
}

function mergeSort(a) {
  if (a.length <= 1) return [...a];
  const mid = Math.floor(a.length / 2);
  const left = mergeSort(a.slice(0, mid));
  const right = mergeSort(a.slice(mid));
  const result = [];
  let i = 0, j = 0;
  while (i < left.length && j < right.length) {
    result.push(left[i] <= right[j] ? left[i++] : right[j++]);
  }
  return result.concat(left.slice(i), right.slice(j));
}

function quickSort(input) {
  const a = [...input];
  function sort(left, right) {
    if (left >= right) return;
    const pivot = a[right];
    let boundary = left;
    for (let i = left; i < right; i++) {
      if (a[i] < pivot) {
        [a[i], a[boundary]] = [a[boundary], a[i]];
```

```
        boundary++;
      }
    }
    [a[boundary], a[right]] = [a[right], a[boundary]];
    sort(left, boundary - 1);
    sort(boundary + 1, right);
  }
  sort(0, a.length - 1);
  return a;
}

for (const sort of [bubbleSort, selectionSort, insertionSort, mergeSort, quickSort]) {
  console.assert(JSON.stringify(sort([3, 1, 2, 1])) === '[1,1,2,3]');
}
console.assert(JSON.stringify([3, 1, 2].sort((a, b) => a - b)) === '[1,2,3]');
```

정렬	평균 시간	최악 시간	추가 공간	기억할 점
버블	`O(n²)`	`O(n²)`	`O(n)` *	이미 정렬되면 조기 종료로 `O(n)`
선택	`O(n²)`	`O(n²)`	`O(n)` *	교환 횟수가 적음
삽입	`O(n²)`	`O(n²)`	`O(n)` *	거의 정렬되면 빠름
병합	`O(n log n)`	`O(n log n)`	`O(n)`	분할 후 합치기
퀵	`O(n log n)`	`O(n²)`	평균 재귀 `O(log n)`	피벗 선택에 민감

* 위 코드가 원본을 보존하려고 `[...]`로 복사해서 `O(n)`이다. 배열 자체를 수정하면 세 알고리즘의 **작업 공간**은 `O(1)`이다. 퀵 정렬은 최악의 경우 재귀 공간도 `O(n)`이다. JS `sort()`는 숫자 정렬에 비교 함수 `(a, b) => a - b`를 넘겨야 한다.

연습문제

거의 정렬된 작은 배열을 직접 정렬 구현해야 한다면 위 셋(버블·선택·삽입) 중 무엇이 적합한가?

풀이: 삽입 정렬. 제자리에서 움직여야 하는 값이 적으면 거의 `O(n)`에 가깝다. 단순히 결과만 필요하면 내장 `sort`가 먼저다.

5. 이분 탐색

한 줄 핵심

정렬된 범위의 가운데를 보고, 답이 없는 절반을 버린다. 한 번에 절반씩 줄어 `O(log n)`.

손으로 따라가기

`[2, 4, 7, 9, 12]`에서 `9` 찾기: 가운데 인덱스 2의 값 `7`은 작다 → 왼쪽 절반 버림 → 인덱스 3의 값 `9`를 찾는다. 정렬되지 않았다면 "왼쪽을 버려도 된다"는 근거가 없어서 이 방법은 실패한다.

실행 코드

```
function binarySearch(a, target) {
  let left = 0, right = a.length - 1;
  while (left <= right) {
    const mid = left + Math.floor((right - left) / 2);
    if (a[mid] === target) return mid;
    if (a[mid] < target) left = mid + 1;
    else right = mid - 1;
  }
  return -1;
}

// target 이상인 값의 첫 위치. 중복 원소에도 유용하다.
function lowerBound(a, target) {
  let left = 0, right = a.length;
  while (left < right) {
    const mid = left + Math.floor((right - left) / 2);
    if (a[mid] < target) left = mid + 1;
    else right = mid;
  }
  return left;
}

console.assert(binarySearch([2, 4, 7, 9, 12], 9) === 3);
console.assert(binarySearch([2, 4, 7], 8) === -1);
console.assert(lowerBound([1, 2, 2, 2, 4], 2) === 1);
```

`binarySearch`는 중복값 중 **어느 하나**를 찾는다. 첫 번째 위치가 필요하면 `lowerBound`를 쓴다. 인덱스 `a.length`가 나오면 조건을 만족하는 원소가 없다는 뜻이다.

연습문제

`[1, 3, 3, 5]`에서 `lowerBound(a, 4)`는?

풀이: 4 이상인 첫 값은 `5`, 인덱스는 `3`이다.

6. 깊이 우선 탐색(DFS)과 너비 우선 탐색(BFS)

한 줄 핵심

- **DFS**: 한 길을 끝까지 간 뒤 돌아온다. 재귀 또는 스택.
- **BFS**: 가까운 곳부터 차례로 간다. 큐. 모든 간선의 비용이 같을 때 최단 간선 수를 찾는다.

손으로 따라가기

그래프 `A: [B,C]`, `B: [D]`, `C: [D]`, `D: []`에서 DFS는 `A → B → D → C`, BFS는 `A → B → C → D`다. `D`는 두 경로에서 도달할 수 있으므로 **방문 표시**가 없으면 중복 방문한다. BFS는 큐에 **넣을 때** 방문 표시를 해야 중복으로 들어가지 않는다.

실행 코드

```
const graph = {
  A: ['B', 'C'], B: ['D'], C: ['D'], D: [],
};

function dfs(graph, start) {
  const seen = new Set();
  const order = [];
  function visit(node) {
    if (seen.has(node)) return;
    seen.add(node);
    order.push(node);
    for (const next of graph[node] ?? []) visit(next);
  }
  visit(start);
  return order;
}

function bfs(graph, start) {
  const seen = new Set([start]);
  const queue = [start];
  const order = [];
  for (let head = 0; head < queue.length; head++) {
    const node = queue[head];
    order.push(node);
    for (const next of graph[node] ?? []) {
      if (seen.has(next)) continue;
      seen.add(next); // 큐에 넣는 순간 표시
      queue.push(next);
    }
  }
  return order;
}

console.assert(dfs(graph, 'A').join('') === 'ABDC');
console.assert(bfs(graph, 'A').join('') === 'ABCD');
```

인접 리스트에서 각 정점과 간선을 한 번씩 확인하므로 둘 다 `O(V+E)` 시간, 방문 집합까지 포함해 `O(V)` 공간이다. 재귀 DFS는 깊은 그래프에서 JS 호출 스택 한도를 넘길 수 있다. 그때는 명시적 스택을 쓴다.

연습문제

모든 통로의 이동 시간이 1인 미로에서 시작점부터 출구까지 **최소 이동 횟수**를 찾으려면?

풀이: BFS. 시작점에서 거리 1인 칸, 거리 2인 칸 순으로 처리하므로 처음 출구를 만났을 때 거리가 최소다.

7. 그래프

한 줄 핵심

그래프는 **정점(대상)**과 **간선(연결)**이다. 지도, 친구 관계, 선행 작업을 표현한다.

손으로 따라가기

정점 `1,2,3`에 간선 `1–2`, `1–3`이 있다면 무방향 인접 리스트는 `1:[2,3]`, `2:[1]`, `3:[1]`이다. 간선을 양쪽에 기록한다. 방향 간선 `1→2`라면 `1`의 목록에만 `2`를 넣는다. 가중치가 있으면 이웃을 `{to, cost}`처럼 저장한다.

표현	간선 여부 확인	이웃 전체 순회	공간	언제 편한가
인접 리스트	보통 해당 목록 길이	`O(차수)`	`O(V+E)`	간선이 적거나 탐색할 때
인접 행렬	`O(1)`	`O(V)`	`O(V²)`	간선 존재를 자주 물을 때

실행 코드

```
const edges = [[1, 2], [1, 3]];
const n = 3;
const list = Array.from({ length: n + 1 }, () => []);
const matrix = Array.from({ length: n + 1 }, () => Array(n + 1).fill(0));

for (const [a, b] of edges) {
  list[a].push(b);
  list[b].push(a); // 무방향 그래프라서 양쪽 기록
  matrix[a][b] = matrix[b][a] = 1;
}

console.assert(list[1].join(',') === '2,3');
console.assert(matrix[2][3] === 0);
console.assert(matrix[3][1] === 1);
```

그래프와 트리 차이: 일반 그래프에는 사이클이 있을 수 있고, 연결되지 않은 부분도 있을 수 있다. 트리는 사이클 없는 연결 그래프이므로 정점 `V`개라면 간선은 `V-1`개다.

연습문제

정점 10만 개, 간선 20만 개 그래프를 BFS해야 한다. 인접 리스트와 행렬 중 무엇을 쓰겠는가?

풀이: 인접 리스트. 행렬은 약 `10¹⁰`칸이 필요하지만 리스트는 정점과 간선 수에 비례한다.

8. 트리와 순회

한 줄 핵심

트리는 부모·자식 관계를 가진 **사이클 없는 연결 구조**다. 순회는 각 노드를 어떤 순서로 방문할지 정하는 일이다.

손으로 따라가기

```
    A
   / \
  B   C
 / \
D   E
```

- 전위(나 → 왼쪽 → 오른쪽): `A B D E C`
- 중위(왼쪽 → 나 → 오른쪽): `D B E A C`
- 후위(왼쪽 → 오른쪽 → 나): `D E B C A`

모든 노드를 방문하므로 세 순회 모두 `O(n)` 시간이다. 재귀 호출 공간은 트리 높이 `h`에 비례해 `O(h)`다. **이진 트리**는 자식이 최대 2개인 트리다. **완전 이진 트리**는 마지막 층을 왼쪽부터 채운다. "모든 내부 노드가 자식 둘"인 트리는 보통 **정 이진 트리(full binary tree)**라고 부르며, 모든 층이 꽉 찬 **포화 이진 트리(perfect binary tree)**와 구분한다. 원본 노트는 이 용어가 섞여 있다.

실행 코드

```
const tree = {
  value: 'A',
  left: {
    value: 'B',
    left: { value: 'D', left: null, right: null },
    right: { value: 'E', left: null, right: null },
  },
  right: { value: 'C', left: null, right: null },
};

function traverse(node, order, result = []) {
  if (node === null) return result;
  if (order === 'pre') result.push(node.value);
  traverse(node.left, order, result);
  if (order === 'in') result.push(node.value);
  traverse(node.right, order, result);
  if (order === 'post') result.push(node.value);
  return result;
}

console.assert(traverse(tree, 'pre').join('') === 'ABDEC');
console.assert(traverse(tree, 'in').join('') === 'DBEAC');
console.assert(traverse(tree, 'post').join('') === 'DEBCA');
```

연습문제

자식보다 부모를 **먼저** 처리해야 하는 작업에는 어떤 순회를 쓰는가?

풀이: 전위 순회. 현재 노드를 먼저 방문한 뒤 자식으로 내려간다. 반대로 자식을 모두 처리한 후 부모 결과를 계산한다면 후위 순회다.

9. 이진 탐색 트리(BST)

한 줄 핵심

모든 노드에서 **왼쪽 값 < 현재 값 < 오른쪽 값**을 지킨다. 중위 순회 결과가 오름차순이다.

손으로 따라가기

`8, 3, 10, 6`을 차례로 넣으면 `3`은 `8`의 왼쪽, `10`은 오른쪽, `6`은 `3`의 오른쪽에 간다. `6` 검색은 `8`과 비교해 왼쪽, `3`과 비교해 오른쪽으로 간다. 삭제할 노드의 자식이 **0개면 제거**, **1개면 그 자식으로 대체**, **2개면 오른쪽 서브트리의 최솟값으로 대체**한 뒤 그 최솟값 노드를 삭제한다.

실행 코드

```
class BSTNode {
  constructor(value) {
    this.value = value;
    this.left = null;
    this.right = null;
  }
}

function insert(root, value) {
  if (root === null) return new BSTNode(value);
  if (value < root.value) root.left = insert(root.left, value);
  else if (value > root.value) root.right = insert(root.right, value);
  return root; // 같은 값은 넣지 않는다.
}

function contains(root, value) {
  while (root !== null) {
    if (value === root.value) return true;
    root = value < root.value ? root.left : root.right;
  }
  return false;
}

function remove(root, value) {
  if (root === null) return null;
  if (value < root.value) root.left = remove(root.left, value);
  else if (value > root.value) root.right = remove(root.right, value);
  else {
    if (root.left === null) return root.right;
    if (root.right === null) return root.left;
    let successor = root.right;
    while (successor.left !== null) successor = successor.left;
    root.value = successor.value;
    root.right = remove(root.right, successor.value);
  }
  return root;
}

let root = null;
for (const value of [8, 3, 10, 1, 6, 14]) root = insert(root, value);
console.assert(contains(root, 6));
root = remove(root, 3); // 자식 2개인 노드 삭제
console.assert(!contains(root, 3) && contains(root, 6));
root = remove(root, 8); // 루트 삭제: 반환값으로 root를 갱신
console.assert(!contains(root, 8));
```

시간은 `O(트리 높이)`다. 균형이 잡히면 `O(log n)`, 정렬된 값을 차례로 넣어 한쪽으로 치우치면 `O(n)`이다. 일반 BST가 항상 빠른 것은 아니다. 이 코드는 중복값을 무시한다. 중복을 허용하려면 별도 규칙이 필요하다.

연습문제

BST에 `1,2,3,4`를 순서대로 넣으면 검색 시간은?

풀이: 노드가 오른쪽으로 한 줄로 이어진다. `4`를 찾는 데 네 노드를 확인하므로 일반화하면 최악 `O(n)`이다.

10. 우선순위 큐와 힙

한 줄 핵심

우선순위 큐는 **중요한 값부터** 꺼낸다. 최소 힙으로 만들면 가장 작은 값이 루트에 있고 삽입·삭제가 `O(log n)`이다.

손으로 따라가기

`[3, 7, 5]`가 최소 힙일 때 `1`을 끝에 넣으면 `[3,7,5,1]`. 부모 `7`보다 작아 교환 → `[3,1,5,7]`; 다시 부모 `3`보다 작아 교환 → `[1,3,5,7]`. 최솟값 `1`을 꺼내면 마지막 `7`을 맨 위로 옮기고 더 작은 자식과 내려가며 교환한다.

배열의 인덱스 `i`에서 부모는 `Math.floor((i-1)/2)`, 왼쪽 자식은 `2i+1`, 오른쪽 자식은 `2i+2`. **힙은 전체가 정렬된 배열이 아니다.** 부모·자식 관계만 지킨다.

실행 코드

```
class MinHeap {
  constructor() { this.a = []; }
  peek() { return this.a[0]; }
  push(value) {
    const a = this.a;
    a.push(value);
    let i = a.length - 1;
    while (i > 0) {
      const parent = Math.floor((i - 1) / 2);
      if (a[parent] <= a[i]) break;
      [a[parent], a[i]] = [a[i], a[parent]];
      i = parent;
    }
  }
  pop() {
    const a = this.a;
    if (a.length === 0) return undefined;
    const min = a[0];
    const last = a.pop();
    if (a.length === 0) return min;
    a[0] = last;
    let i = 0;
    while (2 * i + 1 < a.length) {
      let child = 2 * i + 1;
      if (child + 1 < a.length && a[child + 1] < a[child]) child++;
      if (a[i] <= a[child]) break;
      [a[i], a[child]] = [a[child], a[i]];
      i = child;
    }
    return min;
  }
}

const heap = new MinHeap();
for (const value of [3, 7, 5, 1]) heap.push(value);
console.assert(heap.peek() === 1);
console.assert([heap.pop(), heap.pop(), heap.pop(), heap.pop()].join(',') === '1,3,5,7');
console.assert(heap.pop() === undefined);
```

연습문제

최소 힙 배열이 `[2, 5, 3, 9, 7]`이면 다음 `pop()` 결과는? 남은 값의 순서는?

풀이: `2`가 나온다. 마지막 `7`을 루트로 옮긴 뒤 더 작은 자식 `3`과 교환하면 `[3,5,7,9]`가 된다. 힙 배열이므로 전체 오름차순일 필요는 없다.

11. 동적 프로그래밍(DP)

한 줄 핵심

같은 작은 문제를 여러 번 푼다면 **답을 저장해 재사용**한다. 핵심은 `상태 → 점화식 → 초기값 → 계산 순서`다.

손으로 따라가기

피보나치 `F(n)=F(n-1)+F(n-2)`에서 `F(5)`를 구하려고 `F(4)`와 `F(3)`을 풀면, `F(4)` 안에서도 `F(3)`이 또 나온다. 계산한 `F(3)`을 저장하면 중복 작업이 사라진다. 여기서는 `F(0)=0`, `F(1)=1`로 정의한다. 원본 노트의 "첫째·둘째 항이 1" 표현은 `1,1,2,...`처럼 **1부터 세는 방식**이고 예시 코드는 `0,1,1,...`처럼 **0부터 세는 방식**이다. 둘 다 가능하지만 문제에서 인덱스 정의를 확인해야 한다.

실행 코드

```
// Top-down: 필요한 값부터 재귀로 구하고 저장
function fibTopDown(n) {
  const memo = [0, 1];
  function solve(k) {
    if (memo[k] !== undefined) return memo[k];
    memo[k] = solve(k - 1) + solve(k - 2);
    return memo[k];
  }
  return solve(n);
}

// Bottom-up: 작은 값부터 순서대로 계산
function fibBottomUp(n) {
  if (n <= 1) return n;
  let prev2 = 0, prev1 = 1;
  for (let i = 2; i <= n; i++) {
    [prev2, prev1] = [prev1, prev2 + prev1];
  }
  return prev1;
}

console.assert(fibTopDown(7) === 13);
console.assert(fibBottomUp(7) === 13);

// 추가 예: 한 번에 계단 1칸 또는 2칸, n칸에 도달하는 방법 수
function waysToClimb(n) {
  if (n < 0) return 0;
  const ways = Array(n + 1).fill(0);
  ways[0] = 1; // 아무것도 하지 않는 방법 하나
  for (let i = 1; i <= n; i++) {
    ways[i] = ways[i - 1] + (i >= 2 ? ways[i - 2] : 0);
  }
  return ways[n];
}
console.assert(waysToClimb(4) === 5);
```

두 피보나치 구현은 `O(n)` 시간이다. Top-down은 메모와 호출 스택으로 `O(n)` 공간, Bottom-up은 위 코드처럼 두 값만 저장하면 `O(1)` 공간이다. DP를 무조건 재귀로 시작할 필요는 없다. **점화식이 무엇인지** 먼저 찾는다. 계단 문제에서는 마지막 이동이 1칸 또는 2칸이므로 `ways[i] = ways[i-1] + ways[i-2]`다.

연습문제

한 번에 1칸 또는 2칸 오를 수 있을 때 3칸에 도달하는 방법은 몇 가지인가?

풀이: `1+1+1`, `1+2`, `2+1`의 3가지. `ways[3]=ways[2]+ways[1]=2+1=3`이다.

12. 소수

한 줄 핵심: 소수는 1과 자기 자신만 약수인 **2 이상의 정수**다. 어떤 수 `n`이 합성수라면 `√n` 이하인 약수가 적어도 하나 있다.

손으로 따라가기: `36 = 4 × 9`다. 약수 쌍의 두 수가 모두 `√36 = 6`보다 클 수는 없다. 그래서 `2`부터 `i*i <= n`까지만 나눠 보면 된다. `1`은 소수가 아니다. `2`는 소수다.

```
function isPrime(n) {
  if (!Number.isInteger(n) || n < 2) return false;
  for (let i = 2; i * i <= n; i++) {
    if (n % i === 0) return false;
  }
  return true;
}

console.assert(!isPrime(1) && isPrime(2));
console.assert(isPrime(29) && !isPrime(49));

// 추가: 0부터 limit까지 모든 소수를 반복 조회할 때
function sieve(limit) {
  const prime = Array(limit + 1).fill(true);
  prime[0] = prime[1] = false;
  for (let p = 2; p * p <= limit; p++) {
    if (!prime[p]) continue;
    for (let multiple = p * p; multiple <= limit; multiple += p) {
      prime[multiple] = false;
    }
  }
  return prime;
}

console.assert([2, 3, 5, 7].every(x => sieve(10)[x]));
```

한 수 판별은 `O(√n)` 시간, 추가 공간 `O(1)`이다. `2`부터 `n/2`까지 보는 방법도 빅오로는 여전히 `O(n)`이다. 여러 수를 한꺼번에 조회한다면 에라토스테네스의 체를 만들며, 전처리는 `O(N log log N)` 시간과 `O(N)` 공간이다.

연습문제: `49`의 소수 판별에서 왜 `7`까지만 확인하면 되는가?
풀이: `√49=7`이고 `49=7×7`이다. `7`에서 나누어떨어져 합성수임을 알 수 있다.

13. 누적합과 구간합

한 줄 핵심: 같은 배열의 구간 합을 여러 번 물으면 **앞에서부터의 합을 저장**하고 두 값의 차이로 답한다.

1차원 손으로 따라가기: `A=[1,3,10,6]`, `S=[0,1,4,14,20]`. `S[k]`는 `A[0]`부터 `A[k-1]`까지의 합이다. `A[1]~A[3]`은 전체 앞부분 `S[4]=20`에서 필요 없는 앞부분 `S[1]=1`을 빼서 `19`다. 공식은 양끝 포함 `sum(L,R)=S[R+1]-S[L]`.

```
function prefix1D(a) {
  const s = [0];
  for (const value of a) s.push(s.at(-1) + value);
  return s;
}
function rangeSum(s, left, right) {
  return s[right + 1] - s[left]; // 양끝 포함
}

const s = prefix1D([1, 3, 10, 6]);
console.assert(s.join(',') === '0,1,4,14,20');
console.assert(rangeSum(s, 1, 3) === 19);
```

전처리는 `O(n)`, 각 질문은 `O(1)`이다. 빈 배열도 `S=[0]`으로 처리된다. 배열 값이 **변경될 때마다** 누적합을 다시 만들어야 하므로 이 방식은 값이 고정되고 질문이 많은 경우에 적합하다.

2차원 손으로 따라가기: 아래 표의 `(행 1, 열 1)`부터 `(행 2, 열 2)`까지의 합은 `5+6+8+9=28`이다.

```
1 2 3
4 5 6
7 8 9
```

`P[r+1][c+1]`을 `(0,0)`에서 `(r,c)`까지 직사각형의 합으로 둔다. 계산할 때 **위쪽 합 + 왼쪽 합 - 겹친 부분 + 현재 칸**이다. 질문할 때는 **큰 직사각형 - 위쪽 - 왼쪽 + 두 번 뺀 겹침**이다.

```
function prefix2D(a) {
  if (a.length === 0) return [[0]];
  const rows = a.length, cols = a[0].length;
  if (!a.every(row => row.length === cols)) throw new Error('직사각형 배열 필요');
  const p = Array.from({ length: rows + 1 }, () => Array(cols + 1).fill(0));
  for (let r = 0; r < rows; r++) {
    for (let c = 0; c < cols; c++) {
      p[r + 1][c + 1] = p[r][c + 1] + p[r + 1][c] - p[r][c] + a[r][c];
    }
  }
  return p;
}

function rectangleSum(p, r1, c1, r2, c2) {
  return p[r2 + 1][c2 + 1] - p[r1][c2 + 1]
       - p[r2 + 1][c1] + p[r1][c1];
}

const p = prefix2D([[1,2,3], [4,5,6], [7,8,9]]);
console.assert(rectangleSum(p, 1, 1, 2, 2) === 28);
console.assert(rectangleSum(p, 0, 0, 2, 2) === 45);
```

2차원 전처리는 `O(행×열)`, 각 직사각형 질문은 `O(1)`이다. 행·열 앞에 0을 한 줄씩 추가하면 경계 조건을 따로 나눌 필요가 없다.

연습문제: `A=[2,5,1,4]`에서 `A[1]~A[2]`의 합을 누적합으로 구하라.
풀이: `S=[0,2,7,8,12]`, `S[3]-S[1]=8-2=6`이다.

마지막 점검: 문제에서 방법 고르기

문제의 단서	먼저 떠올릴 방법
최근 항목부터 되돌리기, 괄호	스택
들어온 순서대로 처리	큐
정렬된 데이터에서 값 찾기	이분 탐색
연결 관계를 모두 방문	DFS/BFS
같은 비용의 간선으로 최단 이동 수	BFS
가장 작은 값부터 반복해서 꺼내기	최소 힙
작은 문제의 답이 반복 등장	DP
고정 배열의 구간 합을 여러 번 질문	누적합
한 수가 소수인지 확인	$\sqrt{n}$까지만 나눠 보기

문제를 풀 때는 **입력 크기** → **필요한 연산** → **허용 시간** → **자료구조** 순서로 판단한다. 예컨대 `n=100,000`인데 모든 쌍을 비교하는 $O(n^2)$ 방법은 보통 위험하다. 코드보다 먼저 작은 예시를 손으로 따라가며 불변 조건을 확인한다.